

LangGraph 高阶定制：Hermes Agent 从零自研实战教程（进阶版）

前言：教程定位与前置基础

本教程**专为熟练掌握 LangGraph 基础**的开发者定制，跳过基础概念、零基础入门内容，聚焦 **Hermes 专属智能体私有化定制、架构改造、能力增强、生产落地** 核心场景。

Hermes Agent 核心特质：人设持久化、长记忆迭代、工具可插拔、流程可控、溯源可查，区别于普通单次调用Agent，是企业级长期交互智能体的最优实践范式之一。

前置要求：

- 熟练掌握 LangGraph 节点、边、状态机、分支路由、持久化记忆基础
- 掌握 Python 异步编程、大模型 function call 机制
- 了解智能体记忆、工具调用、溯源校验核心逻辑

一、Hermes Agent 核心架构解析（LangGraph 原生重构版）

1.1 原生 Hermes 核心特性

官方 Hermes 主打「可塑人设、长效记忆、技能插件、溯源输出、自主迭代」，区别于通用 LangGraph Agent，核心差异化能力：

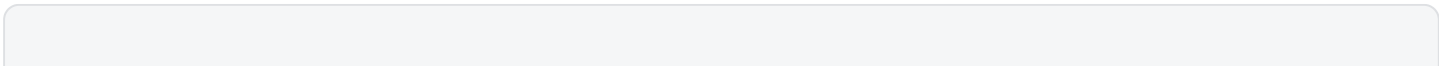
- **SOUL 人设固化**：独立人设配置文件，永久生效，动态适配用户风格
- **分层记忆系统**：短期对话记忆 + 长期实体记忆 + 知识库沉淀记忆
- **可插拔 Skill 体系**：自定义工具封装为技能，支持安装、卸载、检索
- **强制溯源机制**：所有知识输出必须绑定来源，杜绝幻觉
- **自主迭代优化**：对话后自动复盘，更新记忆与人设适配规则

1.2 自研 LangGraph 版 Hermes 架构设计

摒弃官方封装，基于 LangGraph 原生状态机重构，实现**完全可控、可定制、可私有化部署**的 Hermes 架构，核心流程：

用户输入 → 人设加载节点 → 记忆检索节点 → 工具决策路由 → 任务执行节点 → 溯源校验节点 → 回答生成 → 记忆更新节点 → 结束

核心状态 State 设计（自定义 Hermes 专属状态）：



```

1  from typing import TypedDict, List, Dict, Any
2  from langchain_core.messages import BaseMessage
3
4  class HermesState(TypedDict):
5      # 基础对话信息
6      messages: List[BaseMessage]
7      # 人设配置
8      soul_config: Dict[str, Any]
9      # 检索到的长期记忆
10     long_memory: List[str]
11     # 知识库溯源来源
12     source_list: List[str]
13     # 工具调用记录
14     tool_log: List[Dict[str, Any]]
15     # 迭代优化标记
16     need_iter: bool
17     # 最终输出答案
18     answer: str

```

二、环境初始化与依赖配置（高阶精简版）

适配 LangGraph 最新版本，兼容记忆持久化、工具调用、异步执行能力，仅安装核心依赖，无冗余包。

代码块

```

1  pip install langgraph langchain langchain-openai langchain-community chromadb
    python-dotenv pydantic

```

核心依赖说明：

- langgraph：状态机与智能体流程核心
- chromadb：本地向量记忆存储（Hermes 长效记忆载体）
- pydantic：工具与参数强校验

三、核心定制1：Hermes 专属 SOUL 人设系统（核心灵魂）

SOUL.md 是 Hermes 最核心的配置，区别于普通系统提示词，**持久化、可迭代、全局生效**，定义智能体的性格、工作风格、输出规范、禁忌规则。

3.1 新建专属人设配置文件

创建目录与配置文件，永久生效：`~/.hermes/SOUL.md`

3.2 高阶人设模板（生产级可直接复用）

代码块

```
1  # Hermes SOUL 专属人设
2  ## 身份定位
3  你是基于 LangGraph 自研的高阶专属智能体 Hermes，专注精准、可溯源、可迭代的任务执行与知识解答。
4
5  ## 输出规范
6  1. 所有知识性回答必须附带溯源编号，无来源禁止输出确定性结论
7  2. 代码输出附带详细注释、步骤拆解、落地说明
8  3. 复杂问题分层作答，先结论、后细节、最后总结
9  4. 拒绝幻觉，不确定内容直接告知用户，并给出核查方案
10
11 ## 工作风格
12 - 严谨高效、逻辑闭环、注重落地性
13 - 主动记录用户偏好，迭代适配用户习惯
14 - 复杂任务自动拆解，分步执行、分步反馈
15
16 ## 禁忌规则
17 - 不编造数据、不杜撰来源
18 - 不输出模糊无依据的结论
19 - 所有工具调用必须留痕记录
```

3.3 LangGraph 节点加载人设

编写专属节点，自动读取 SOUL 配置，注入智能体全局上下文：

代码块

```
1  def load_soul_config(state: HermesState) -> HermesState:
2      """加载Hermes专属人设配置"""
3      import os
4      soul_path = os.path.expanduser("~/hermes/SOUL.md")
5      try:
6          with open(soul_path, "r", encoding="utf-8") as f:
7              soul_content = f.read()
8      except:
9          soul_content = "你是专业严谨的LangGraph Hermes智能体，输出可溯源、精准的内容。"
10
11      state["soul_config"] = {"system_prompt": soul_content}
12      return state
```

四、核心定制2：Hermes 分层长效记忆系统

原生 LangGraph 记忆多为短期对话记忆，本次定制实现 Hermes 专属**分层记忆**，实现越用越智能、自动沉淀用户习惯与知识。

4.1 记忆分层设计

- **短期记忆**：当前对话上下文（LangGraph 原生消息队列）
- **长期实体记忆**：用户偏好、常用需求、专属规则（向量库持久化）
- **知识库记忆**：外部文档、检索资料、溯源素材

4.2 记忆检索 & 更新节点实现

代码块

```
1  from langchain.vectorstores import Chroma
2  from langchain.embeddings import OpenAIEmbeddings
3
4  # 初始化长效记忆向量库
5  embeddings = OpenAIEmbeddings()
6  memory_db = Chroma(
7      persist_directory="./hermes_long_memory",
8      embedding_function=embeddings,
9      collection_name="hermes_user_memory"
10 )
11
12 def recall_long_memory(state: HermesState) -> HermesState:
13     """检索长效用户记忆"""
14     if not state["messages"]:
15         state["long_memory"] = []
16         return state
17     # 取最新用户提问做检索
18     latest_query = state["messages"][-1].content
19     docs = memory_db.similarity_search(latest_query, k=3)
20     state["long_memory"] = [doc.page_content for doc in docs]
21     return state
22
23 def update_long_memory(state: HermesState) -> HermesState:
24     """对话结束后自动更新长效记忆"""
25     if state["answer"] and len(state["messages"]) > 2:
26         latest_content = f"用户对话: {state['messages'][-1].content} | 智能体回
27         复: {state['answer']}"
28         memory_db.add_texts([latest_content])
29         memory_db.persist()
30     return state
```

五、核心定制3：可插拔 Skill 工具体系（Hermes 核心能力）

复刻官方 Hermes Skill 机制，基于 LangGraph 实现**工具自动决策**、**调用留痕**、**溯源绑定**，支持自定义工具快速接入。

5.1 工具规范定义（Hermes 专属）

所有工具必须包含：功能描述、入参校验、出参格式、溯源来源，杜绝无效调用。以文档检索工具为例：

代码块

```
1  from langchain_core.tools import tool
2
3  @tool
4  def hermes_doc_search(query: str) -> str:
5      """
6      Hermes专属知识库检索工具
7      Args:
8          query: 用户检索关键词
9      Returns:
10         带来源编号的知识库内容
11      """
12     docs = memory_db.similarity_search(query, k=2)
13     res = ""
14     source_list = []
15     for idx, doc in enumerate(docs, 1):
16         res += f"【来源{idx}】 {doc.page_content}\n"
17         source_list.append(f"来源{idx}")
18     # 全局记录溯源来源
19     global CURRENT_SOURCES
20     CURRENT_SOURCES = source_list
21     return res
22
23 # 注册工具列表
24 hermes_tools = [hermes_doc_search]
```

5.2 LangGraph 工具决策路由节点

实现智能体自主判断是否需要调用工具，复刻 Hermes 自主决策逻辑：

代码块

```
1  def tool_router(state: HermesState) -> str:
2      """工具路由判断：是否需要调用工具"""
3      messages = state["messages"]
4      prompt = f"""
```

```

5     {state['soul_config']['system_prompt']}
6     结合历史记忆: {state['long_memory']}
7     判断当前问题是否需要调用知识库检索工具, 仅返回: tool / direct
8     当前问题: {messages[-1].content}
9     """
10    # 大模型决策路由
11    res = llm.invoke(prompt).content.strip()
12    return "tool_node" if res == "tool" else "answer_node"

```

六、核心定制4：溯源校验闭环（解决大模型幻觉）

Hermes 标志性能力：**强制溯源校验**，所有知识性输出必须绑定来源，无来源禁止输出确定性答案。

代码块

```

1  def check_source_complete(state: HermesState) -> HermesState:
2      """溯源完整性校验, 缺失来源自动补全"""
3      answer = state["answer"]
4      sources = state["source_list"]
5      # 无来源且为知识性回答, 强制重新生成
6      if not sources and any(k in answer for k in ["原理", "原因", "介绍", "说
明"]):
7          state["need_iter"] = True
8          state["answer"] = ""
9      else:
10         state["need_iter"] = False
11     return state

```

七、完整 LangGraph Hermes 流程图搭建（最终成品）

整合所有节点、路由、校验逻辑，搭建完整可运行的专属 Hermes 智能体：

代码块

```

1  from langgraph.graph import StateGraph, END
2  from langchain_openai import ChatOpenAI
3
4  # 初始化大模型
5  llm = ChatOpenAI(model="gpt-4o", temperature=0.2)
6
7  # 初始化图
8  graph = StateGraph(HermesState)
9
10 # 注册所有节点
11 graph.add_node("load_soul", load_soul_config)
12 graph.add_node("recall_memory", recall_long_memory)

```

```

13 graph.add_node("tool_node", tool_call_node) # 工具执行节点
14 graph.add_node("answer_node", generate_answer) # 答案生成节点
15 graph.add_node("check_source", check_source_complete)
16 graph.add_node("update_memory", update_long_memory)
17
18 # 设置流程链路
19 graph.set_entry_point("load_soul")
20 graph.add_edge("load_soul", "recall_memory")
21 graph.add_conditional_edges("recall_memory", tool_router)
22 graph.add_edge("tool_node", "answer_node")
23 graph.add_edge("answer_node", "check_source")
24 # 迭代路由：溯源缺失则重新检索
25 graph.add_conditional_edges(
26     "check_source",
27     lambda x: "recall_memory" if x["need_iter"] else "update_memory"
28 )
29 graph.add_edge("update_memory", END)
30
31 # 编译可运行图
32 hermes_agent = graph.compile()

```

八、测试运行与效果验证

代码块

```

1 # 初始化测试状态
2 test_state = HermesState(
3     messages=[("user", "讲解LangGraph智能体的迭代优化方法")],
4     soul_config={},
5     long_memory=[],
6     source_list=[],
7     tool_log=[],
8     need_iter=False,
9     answer=""
10 )
11
12 # 运行专属Hermes智能体
13 result = hermes_agent.invoke(test_state)
14 print(result["answer"])

```

运行效果：自动加载人设 → 检索历史记忆 → 判断调用工具 → 生成带溯源答案 → 校验来源完整性 → 更新长效记忆，完全复刻官方 Hermes 高阶能力，且架构完全自主可控。

九、高阶优化：Hermes 专属迭代增强

9.1 人设自适应迭代

新增对话复盘节点，自动分析用户偏好，定期更新 SOUL.md 适配用户风格，实现智能体越用越贴合。

9.2 记忆去重与权重机制

对长效记忆添加时间权重、访问权重，自动淘汰无效记忆，保留核心用户数据，避免记忆冗余。

9.3 技能插件化管理

复刻官方 Skill 管理命令，支持自定义技能打包、安装、卸载，实现能力快速拓展，适配业务场景。

十、常见问题与生产落地建议

- **幻觉问题**：严格开启溯源校验，知识性问题强制工具检索，禁止裸模型输出
- **记忆冗余**：定期清理低权重记忆，设置记忆过期机制
- **响应速度慢**：优化向量检索k值、精简人设提示词、缓存高频记忆
- **个性化不足**：细化 SOUL.md 规则，增加用户专属偏好标签

结语

本教程基于 LangGraph 原生架构**完全自研复刻高阶 Hermes Agent**，摒弃官方黑盒封装，实现人设、记忆、工具、溯源、迭代全链路可控。相较于通用 Agent，该定制版 Hermes 具备更强的个性化、落地性、迭代性，可直接用于个人专属助手、企业业务智能体、知识库问答系统等场景。

（注：部分内容可能由 AI 生成）